

A rule r in SCLRAM is denoted $p \leftarrow e$, where predicate p is the rule head and expression e is the rule body. An unordered set of rules combined form a stratum s , and a sequence of strata $s_1; \dots; s_n$ constitutes a SCLRAM program. Rules in the same stratum have distinct head predicates. Denoting the set of head predicates in stratum s by P_s , we also require $P_{s_i} \cap P_{s_j} = \emptyset$ for all $i \neq j$ in a program. Stratified negation and aggregation from the surface language is enforced as syntax restrictions in SCLRAM: within a rule in stratum s_i , if a relational predicate p is used under aggregation (γ) or right-hand-side of difference ($-$), that predicate p cannot appear in P_{s_j} if $j \geq i$.

We next define the semantic domains in Fig. 6 which are used subsequently to define the semantics of SCLRAM. A tuple u is either a constant or a sequence of tuples. A fact $p(u) \in \mathbb{F}$ is a tuple u named under a relational predicate p . Tuples and facts can be tagged, forming *tagged tuples* ($t :: u$) and *tagged facts* ($t :: p(u)$). Given a set of tagged tuples U_T , we say $U_T \models u$ iff there exists a t such that $t :: u \in U_T$. A set of tagged facts form a database F_T . We use bracket notation $F_T[p]$ to denote the set of tagged facts in F_T under predicate p .

4.3 Operational Semantics of SCLRAM

We now present the operational semantics for our core fragment of SCLRAM in Fig. 7. A SCLRAM program $\bar{s}$ takes as input an *extensional database* (EDB) F_T , and returns an *intentional database* (IDB) $F'_T = \llbracket \bar{s} \rrbracket(F_T)$. The semantics is conditioned on the underlying provenance T . We call this *tagged semantics*, as opposed to the *untagged semantics* found in traditional Datalog.

Basic Relational Algebra. Evaluating an expression in SCLRAM yields a set of tagged tuples according to the rules defined at the top of Fig. 7. A predicate p evaluates to all facts under that predicate in the database. Selection filters tuples that satisfy condition β , and projection transforms tuples according to mapping α . The mapping function α is partial: it may fail since it can apply foreign functions to values. A tuple in a union $e_1 \cup e_2$ can come from either e_1 or e_2 . In (cartesian) product, each pair of incoming tuples combine and we use $\otimes$ to compute the tag conjunction.

Difference and Negation. To evaluate a difference expression $e_1 - e_2$, there are two cases depending on whether a tuple u evaluated from e_1 appears in the result of e_2 . If it does not, we simply propagate the tuple and its tag to the result (DIFF-1); otherwise, we get $t_1 :: u$ from e_1 and $t_2 :: u$ from e_2 . Instead of erasing the tuple u from the result as in untagged semantics, we propagate a tag $t_1 \otimes (\ominus t_2)$ with u (DIFF-2). In this manner, information is not lost during negation. Fig. 8 compares the evaluations of an example difference expression under different semantics. While the tuple (2, 3) is removed in the outcome with untagged semantics, it remains with the tagged semantics.

Aggregation. Aggregators in SCLRAM are discrete functions g operating on set of (untagged) tuples $U \in \mathcal{U}$. They return a *set* of aggregated tuples to account for aggregators like argmax which can produce multiple outcomes. For example, we have $\text{count}(U) = \{|U|\}$. However, in the probabilistic domain, discrete symbols do not suffice. Given n tagged tuples to aggregate over, each tagged tuple can be turned on or off, resulting in 2^n distinct *worlds*. Each world is a partition of the input set U_T ($|U_T| = n$). Denoting the positive part as X_T and the negative part as $\bar{X}_T = U_T - X_T$, the tag associated with this world is a conjunction of tags in X_T and negated tags in $\bar{X}_T$. Aggregating on this world then involves applying aggregator g on tuples in the positive part X_T . This is inherently exponential if we enumerate all worlds. However, we can optimize over each aggregator and each provenance to achieve better performance. For instance, counting over max-min-prob tagged tuples can be implemented by an $O(n \log(n))$ algorithm, much faster than exponential. Fig. 9 demonstrates a running example and an evaluation of a counting expression under max-min-prob provenance. The resulting count can be 0-9, each derivable by multiple worlds.

Rules and Fixed-Point Iteration. Evaluating a rule $p \leftarrow e$ on database F_T concerns evaluating the expression e and merging the result with the existing facts under predicate p in F_T . The result of evaluating e may contain duplicated tuples tagged by distinct tags, owing to expressions such as

Expression semantics

$$\begin{array}{c}
 \boxed{\alpha : \mathbb{U} \rightarrow \mathbb{U}, \quad \beta : \mathbb{U} \rightarrow \text{Bool}, \quad g : \mathcal{U} \rightarrow \mathcal{U}, \quad \llbracket e \rrbracket : \mathcal{F}_T \rightarrow \mathcal{U}_T} \\
 \frac{t :: p(u) \in F_T}{t :: u \in \llbracket p \rrbracket(F_T)} \text{ (PREDICATE)} \quad \frac{t :: u \in \llbracket e \rrbracket(F_T) \quad \beta(u) = \text{true}}{t :: u \in \llbracket \sigma\beta(e) \rrbracket(F_T)} \text{ (SELECT)} \quad \frac{t :: u \in \llbracket e \rrbracket(F_T) \quad u' = \alpha(u)}{t :: u' \in \llbracket \pi_\alpha(e) \rrbracket(F_T)} \text{ (PROJECT)} \\
 \frac{t :: u \in \llbracket e_1 \rrbracket(F_T) \cup \llbracket e_2 \rrbracket(F_T)}{t :: u \in \llbracket e_1 \cup e_2 \rrbracket(F_T)} \text{ (UNION)} \quad \frac{t_1 :: u_1 \in \llbracket e_1 \rrbracket(F_T) \quad t_2 :: u_2 \in \llbracket e_2 \rrbracket(F_T)}{(t_1 \otimes t_2) :: (u_1, u_2) \in \llbracket e_1 \times e_2 \rrbracket(F_T)} \text{ (PRODUCT)} \\
 \frac{t :: u \in \llbracket e_1 \rrbracket(F_T) \quad \llbracket e_2 \rrbracket(F_T) \not\models u}{t :: u \in \llbracket e_1 - e_2 \rrbracket(F_T)} \text{ (DIFF-1)} \quad \frac{t_1 :: u \in \llbracket e_1 \rrbracket(F_T) \quad t_2 :: u \in \llbracket e_2 \rrbracket(F_T)}{(t_1 \otimes (\ominus t_2)) :: u \in \llbracket e_1 - e_2 \rrbracket(F_T)} \text{ (DIFF-2)} \\
 \frac{X_T \subseteq \llbracket e \rrbracket(F_T) \quad \{t_i :: u_i\}_{i=1}^n = X_T \quad \{\bar{t}_j :: \bar{u}_j\}_{j=1}^m = \llbracket e \rrbracket(F_T) - X_T \quad u \in g(\{u_i\}_{i=1}^n)}{(\bigotimes_{i=1}^n t_i) \otimes (\bigotimes_{j=1}^m (\ominus \bar{t}_j)) :: u \in \llbracket \gamma_g(e) \rrbracket(F_T)} \text{ (AGGREGATE)}
 \end{array}$$

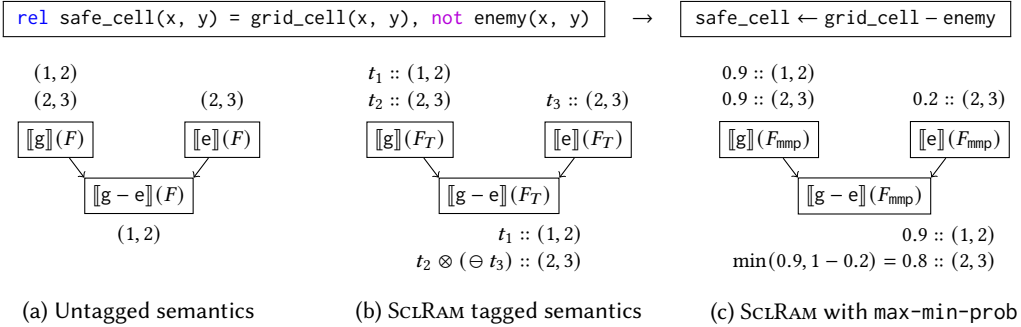
Rule semantics

$$\begin{array}{c}
 \boxed{\langle \cdot \rangle : \mathcal{U}_T \rightarrow \mathcal{U}_T, \quad \llbracket r \rrbracket : \mathcal{F}_T \rightarrow \mathcal{F}_T} \\
 \text{(NORMALIZE)} \quad \langle U_T \rangle = \{(\bigoplus_{i=1}^n t_i) :: u \mid t_1 :: u, \dots, t_n :: u \text{ are all tagged-tuples in } U_T \text{ with the same tuple } u\} \\
 \frac{t^{\text{old}} :: u \in \llbracket p \rrbracket(F_T) \quad \langle \llbracket e \rrbracket(F_T) \rangle \not\models u}{t^{\text{old}} :: p(u) \in \llbracket p \leftarrow e \rrbracket(F_T)} \text{ (RULE-1)} \quad \frac{t^{\text{new}} :: u \in \langle \llbracket e \rrbracket(F_T) \rangle \quad \llbracket p \rrbracket(F_T) \not\models u}{t^{\text{new}} :: p(u) \in \llbracket p \leftarrow e \rrbracket(F_T)} \text{ (RULE-2)} \\
 \frac{t^{\text{old}} :: u \in \llbracket p \rrbracket(F_T) \quad t^{\text{new}} :: u \in \langle \llbracket e \rrbracket(F_T) \rangle}{(t^{\text{old}} \oplus t^{\text{new}}) :: p(u) \in \llbracket p \leftarrow e \rrbracket(F_T)} \text{ (RULE-3)}
 \end{array}$$

Stratum and Program semantics

$$\begin{array}{c}
 \boxed{\text{lfp}^\circ : (\mathcal{F}_T \rightarrow \mathcal{F}_T) \rightarrow (\mathcal{F}_T \rightarrow \mathcal{F}_T), \quad \llbracket s \rrbracket, \llbracket \bar{s} \rrbracket : \mathcal{F}_T \rightarrow \mathcal{F}_T} \\
 \text{(SATURATION)} \quad F_T^{\text{old}} \triangleq F_T^{\text{new}} \text{ iff } \forall t^{\text{new}} :: p(u) \in F_T^{\text{new}}, \exists t^{\text{old}} :: p(u) \in F_T^{\text{old}} \text{ such that } t^{\text{old}} \oplus t^{\text{new}} \\
 \text{(FIXPOINT)} \quad \text{lfp}^\circ(h) = h \circ \dots \circ h = h^n \text{ if there exists a minimum } n > 0, \text{ such that } h^n(F_T) \triangleq h^{n+1}(F_T) \\
 \text{(STRATUM)} \quad \llbracket s \rrbracket = \text{lfp}^\circ(\lambda F_T. (F_T - \bigcup_{p \in P_s} F_T[p]) \cup (\bigcup_{r \in s} \llbracket r \rrbracket(F_T))) \\
 \text{(PROGRAM)} \quad \llbracket \bar{s} \rrbracket = \llbracket s_n \rrbracket \circ \dots \circ \llbracket s_1 \rrbracket, \text{ where } \bar{s} = s_1; \dots; s_n.
 \end{array}$$

Fig. 7. Operational semantics of core fragment of SCLRAM.


 Fig. 8. An example rule adapted from Section 2 is compiled to a SCLRAM rule with difference. g and e are abbreviated relation names. The graphs illustrate the evaluation of expression $g - e$ under different semantics.

union, projection, or aggregation. Therefore, we perform *normalization* on the set to join ($\oplus$) the distinct tags. From here, there are three cases to merge the newly derived tuples ($\langle \llbracket e \rrbracket(F_T) \rangle$) with the previously derived tuples ($\llbracket p \rrbracket(F_T)$). If a fact is present only in the old or the new, we simply propagate the fact to the output. When a tuple u appears in both the old and the new, we propagate the disjunction of the old and new tag ($t^{\text{old}} \oplus t^{\text{new}}$). Combining all cases, we obtain a set of newly tagged facts under predicate p .

Recursion in SCLRAM is realized similar to least fixed point iteration in Datalog [Abiteboul et al. 1995]. The iteration happens on a per-stratum basis to enforce stratified negation and aggregation. Evaluating a single step of stratum s means evaluating all the rules in s and returning the updated

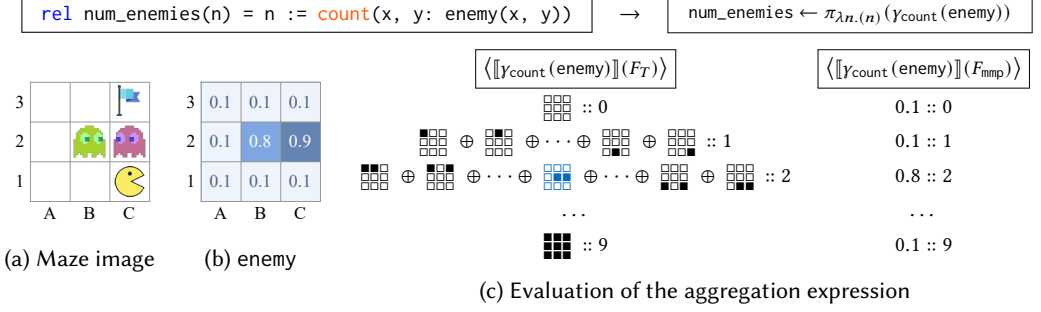


Fig. 9. An example counting enemies in a PacMan maze. Shown above are the Scallop rule and compiled SCLRAM rule with aggregation. (a) and (b) visualize the maze and the content of a probabilistic enemy relation. For example, we have $t_{B2} :: \text{enemy}(B, 2)$ where $t_{B2} = 0.8$. In (c), we show two normalized $\langle \cdot \rangle$ defined in Fig. 7) evaluation results under abstract tagged semantics and with max-min-prob provenance. Each symbol such as $\blacksquare$ represents a world corresponding to our arena ($\blacksquare$: enemy; $\square$: no enemy). A world is a conjunction of 9 tags, e.g., $\blacksquare_{A3} \otimes (\ominus t_{A2}) \otimes \dots \otimes (\ominus t_{C1})$. We mark the correct world $\blacksquare_{B2}$ which yields the answer 2.

<code>rel path(x1,y1,x3,y3) = (edge(x1,y1,x3,y3) or path(x1,y1,x2,y2) and edge(x2,y2,x3,y3)) and not enemy(x3,y3)</code>								
Iteration count i	1	2	3	4	5	6	7	
$t_{C1-C3}^{(i)}$ in $F_T^{(i)}$	–	$\square \uparrow$	(same)	$\square \uparrow \oplus \square \uparrow \oplus \dots \oplus \square \uparrow$	(same)	$\square \uparrow \oplus \square \uparrow \oplus \dots \oplus \square \uparrow$	(same)	
$t_{C1-C3}^{(i)}$ in $F_{\text{mmp}}^{(i)}$	–	0.1	0.1	0.2	0.2	0.9	0.9	
$t_{C1-C3}^{(i)}$ saturated?	–	false	true	false	true	false	true	
$F_{\text{mmp}}^{(i)}$ saturated?	false	false	false	false	false	false	true	

Fig. 10. A demonstration of the fixed-point iteration to check whether actor at C1 can reach C3 without hitting an enemy (Fig. 9a). The Scallop rule to derive this is defined on the top, and we assume bidirectional edges are populated and tagged by 1. Let t_{C1-C3} be the tag associated with $\text{path}(C, 1, C, 3)$. We use a symbol like $\square \uparrow$ to represent a conjunction of negated tags of enemy along the illustrated path, e.g., $\square \uparrow = (\ominus t_{C2}) \otimes (\ominus t_{C3})$. 2nd iter is the first time t_{C1-C3} is derived, but the path $\square \uparrow$ is blocked by an enemy. On 6th iter, the best path $\square \uparrow \oplus \square \uparrow \oplus \dots \oplus \square \uparrow$ is derived in the tag. After that, under the max-min-prob provenance, both the tag t_{C1-C3} and the database F_{mmp} are saturated, causing the iteration to stop. Compared to untagged semantics in Datalog which will stop after 4 iterations, SCLRAM with mmp saturates slower but allowing to explore better reasoning chains.

database. Note that we define a specialized least fixed point operator lfp° , which stops the iteration once the whole database is *saturated*. Fig. 10 illustrates an evaluation involving recursion and database saturation. The whole database saturates on the 7th iteration, and finds the tag representing the optimal path for the PacMan to reach the goal. Termination is not universally guaranteed in SCLRAM due to the presence features such as value creation. But its existence can be proven in a per-provenance basis. For example, it is easy to show that if a program terminates under untagged semantics, then it terminates under tagged semantics with max-min-prob provenance.

4.4 External Interface and Execution Pipeline

Thus far, we have only illustrated the max-min-prob provenance, in which the tags are approximated probabilities. There are other probabilistic provenances with more complex tags such as proof trees or boolean formulae. We therefore introduce, for each provenance T , an *input tag* space I , an *output tag* space O , a *tagging function* $\tau : I \rightarrow T$, and a *recover function* $\rho : T \rightarrow O$. For instance, all probabilistic provenances share the same input and output tag spaces $I = O = [0, 1]$ for a unified interface, while the internal tag spaces T could be different. We call the 4-tuple (I, O, τ, ρ) the